PHPBB 3.0 EXTENSION · SOFTWARE ARCHITECTURE

bbTips

How a BBCode becomes a Wowhead pop-up tooltip

> A server-side text-parsing MOD that rewrites custom BBCodes such as `[item]…[/item]` into Wowhead-shaped anchors, which a client-side widget script then upgrades into rich hover pop-ups.

Package bbdkp / bbTips v1.0.7
Target phpBB 3.0.14 · PHP 5.x
Path development/PHP/phpbb30_mods/bbTips
Prepared 29 July 2026

Contents

1 Executive summary

bbTips lets forum authors reference World of Warcraft game objects with simple BBCodes and have them rendered as colored links that reveal a fully-styled Wowhead tooltip on hover.

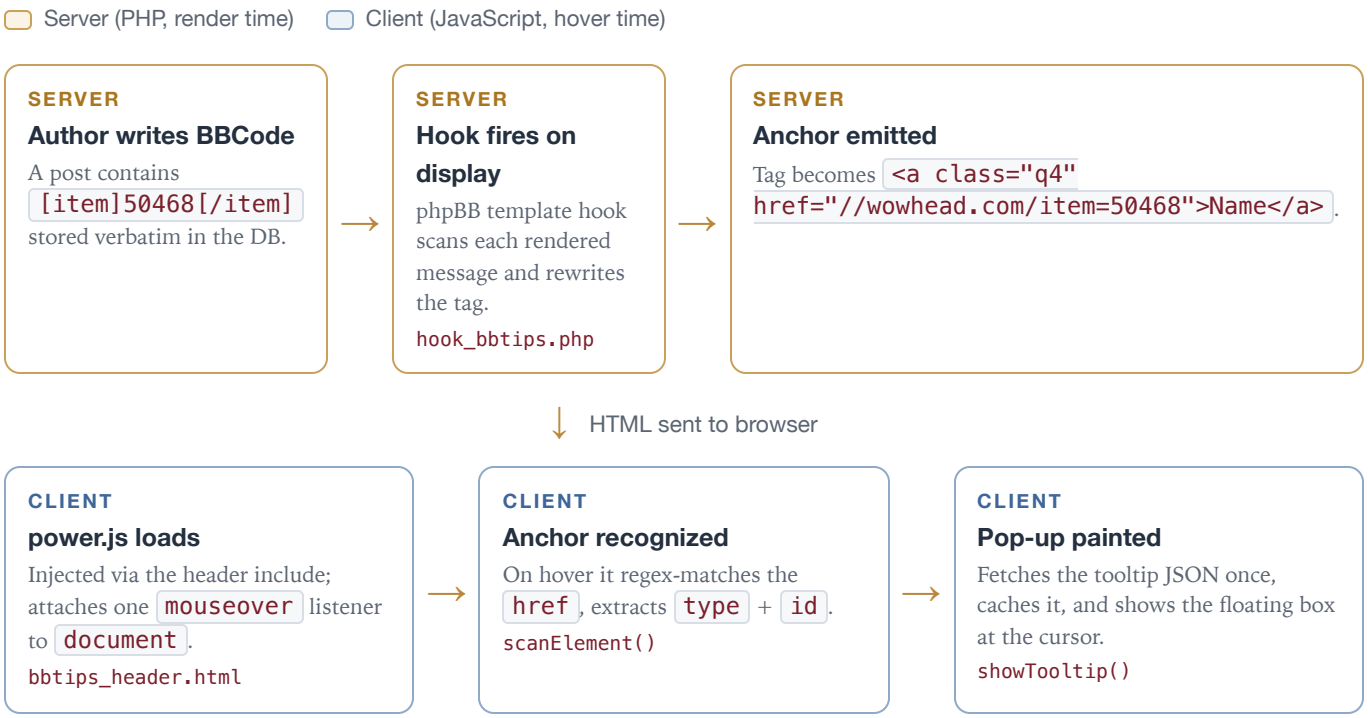
The extension solves this in **two independent stages that never share memory**:

1. **A server-side PHP text filter** runs during page rendering. It hooks into phpBB's template display, scans every post body for bbTips BBCodes, resolves each one against Wowhead (by numeric ID or by name), caches the result in the database, and replaces the tag with a plain `<a href="…wowhead.com/item=NNNN">` anchor built from an HTML template.
2. **A client-side widget script** (`power.js`, Wowhead's own tooltip engine, shipped locally) is injected into every page header. It listens for `mouseover` on the whole document, recognizes any anchor whose URL matches the Wowhead pattern, lazily fetches the tooltip payload, and paints the floating pop-up next to the cursor.

> **Key idea.** The server never produces a pop-up. It only produces a *link that the pop-up engine knows how to recognize*. The visual tooltip is created entirely in the browser at hover time. This clean seam is what makes the design work across posts, previews, news, and the raid planner without special-casing any of them.

2 The two-halves architecture

The contract between the halves is a single convention: an `<a>` tag whose `href` points at `wowhead.com/<type>=<id>`. Everything upstream exists to produce that anchor; everything downstream exists to consume it.



SERVER HALF — RESPONSIBILITIES

- Recognize ~20 BBCode variants (item, spell, quest, craft, itemset, npc, achievement, itemico, itemdkp, wowchar, and their `ptr…` twins).
- Resolve name → ID via Wowhead XML / JSON scraping.
- Persist resolved objects to a cache table so lookups happen at most once.
- Assemble the output anchor from a per-style HTML template.
- Sanitize tag contents against HTML/JS injection.

CLIENT HALF — RESPONSIBILITIES

- Load exactly once per page from the forum header.
- Detect Wowhead-shaped anchors anywhere on the page.
- Lazily request tooltip data only for links actually hovered.
- Cache each tooltip in a JS lookup table for the page's lifetime.
- Render, position, and tear down the floating pop-up DOM.

Local vs. remote mode. A config flag (`bbtips_localjs`, surfaced as `S_BBTIPS_REMOTE`) chooses whether the header loads Wowhead's hosted `power.js` or the copy bundled inside the style. This is the extension's one runtime switch affecting the client half.

3 End-to-end pipeline — a single `[item]`

Follow one tag, `[item]50468[/item]`, from keystroke to pop-up. Stages 1–7 are PHP at render time; stages 8–11 are JavaScript at hover time.

#	Stage	What happens	Where
1	Authoring	User types the BBCode; phpBB stores the raw text.	`phpbb_posts.post_text`
2	Hook trigger	On every page display, the registered template hook runs after the message is rendered.	`hook_bbtips.php → bbtipshooks()`
3	Message sweep	The hook walks `postrow` / `news_row` / preview / raidplanner blocks and calls the parser on each `MESSAGE`.	`bbtips_parser::parse()`
4	Tag match	A regex loop finds `[item]…[/item]` (with or without arguments) up to a cap.	`preg_match loop`
5	Dispatch	The tag name selects a handler class; `bbtips_item` is instantiated with parsed arguments.	`switch($match[1])`
6	Resolve + cache	Cache is checked first; on a miss, Wowhead is queried by XML then JSON, and the result is saved.	`bbtips_item::parse() · dbal.php`
7	Emit anchor	An HTML template ("pattern") is filled with `{link} {name} {qid}` and `str_replace`'d back into the message.	`wowhead_patterns · item.html`
8	Header inject	The rendered page's `<head>` pulls in `power.js` + `basic.js` and the CSS.	`bbtips_header.html`
9	Global listener	`power.js` attaches one `mouseover` handler to the document and waits.	`attachEvent()`
10	Hover match	On hover the anchor's `href` is regex-matched; `type=item`, `id=50468` are extracted.	`scanElement()`
11	Fetch + paint	Tooltip data is fetched once (then cached in JS), and the styled pop-up is shown at the cursor.	`request() → showTooltip()`

Two caches, two layers. The design caches at *both* seams: the PHP side caches the resolved object in the database (so a name is never looked up twice across page loads), and the JS side caches the tooltip payload in an in-memory lookup table (so a link hovered twice fetches once). Neither layer knows about the other.

4 Component & file map

Everything installs under the phpBB root; the two halves live in clearly separated trees — PHP logic under `includes/`, presentation under `styles/`.

```
root/includes/hooks/ hook_bbtips.php # ← ENTRY POINT: registers the template hook
root/includes/bbdkp/bbtips/ # ── SERVER HALF ── bbtips_parser.php # tag scanner + dispatcher
(the orchestrator) bbtips.php # abstract base: URL build, HTTP fetch, wildcards bbtips_item.php
# item / itemico / itemdkp handler bbtips_spell.php bbtips_quest.php bbtips_craft.php
bbtips_itemset.php bbtips_npc.php bbtips_achievement.php wowhead_patterns.php # loads HTML
templates as fill-in patterns dbal.php # bbtips_cache: DB read/write of resolved objects
simple_html_dom.php # DOM scraper for name→ID search pages
root/styles/prosilver/template/bbtips/ # ── CLIENT HALF ── bbtips_header.html # injects
power.js/basic.js + CSS into <head> power.js # Wowhead tooltip engine (mouseover → pop-up)
basic.js # Wowhead $WH DOM/util library power.js needs item.html icon_medium.html spell.html
quest.html … # output patterns root/styles/prosilver/theme/ bbtips.css # pop-up + quality-color
styling root/includes/acp/ · root/language/ # admin config + i18n (en/de/fr)
```

The role each server file plays

File	Class	Responsibility
hook_bbtips.php	–	Registers `bbtipshooks()` on the `('template','display')` hook; bails early if the MOD is not installed or the board is disabled.
bbtips_parser.php	bbtips_parser	Finds each tag, parses its arguments, chooses and instantiates the right handler, sanitizes contents, splices the result back into the message.
bbtips.php	bbtips (abstract)	Shared machinery: build the Wowhead URL, fetch over cURL/fopen/fsockopen, decide the domain/locale, fill template wildcards, emit "not found".
bbtips_item.php	bbtips_item	Concrete handler. Cache lookup → XML lookup → JSON name search → build enhancement string → render pattern.
wowhead_patterns.php	wowhead_patterns	Reads every `*.html` in the style's `bbtips/` folder into a name→markup map used as fill-in templates.
dbal.php	bbtips_cache	Thin SQL wrapper: `getObject()` / `saveObject()` against the cache table (plus gem helpers).

5 Server side — parsing a BBCode

5.1 The hook: where control is intercepted

bbTips does *not* register a normal phpBB BBCode. Instead it registers a **template display hook**. When phpBB is about to render a page, `bbtipshooks()` runs and reaches directly into the compiled template data, rewriting message text just before it is shown.

```
// hook_bbtips.php — registered only if installed & board active
$phpbb_hook->register(array('template', 'display'), 'bbtipshooks');

function bbtipshooks(&$hook, $handle, $include_once = true){
  // pick the right block: normal posts, news, preview, or raid planner
  foreach($template->_tpldata['postrow'] as $key => $val){
    $template->_tpldata['postrow'][$key]['MESSAGE'] =
        $bbtips->parse( $template->_tpldata['postrow'][$key]['MESSAGE'] );
  }
}
```

> **Why a hook and not a BBCode?** A native phpBB BBCode is expanded once at *post-submit* time and frozen into the stored HTML. bbTips needs to (a) reach an external service, (b) cache results, and (c) re-evaluate on *every* view (posts, previews, news, raid planner). A display hook gives it that live, render-time interception across all of those surfaces with a single registration. This also removed the earlier need to edit core `functions.php` (see the 1.0.4 changelog).

5.2 The parser: match → dispatch → splice

`bbtips_parser::parse()` is the orchestrator. It runs a bounded loop, and on each pass:

1. **Matches a tag.** One regex handles argument-less tags (`[item]…[/item]`); a second handles tags with attributes (`[item gems="…" enchant="…"]…[/item]`). The recognized tag names are a fixed pipe-list of ~20 codes.
2. **Parses arguments** (`size=`, `gems=`, `enchant=`, `bonus=`, `realm=`, `lang=`, `mats`, …) into an array.
3. **Dispatches** on the tag name through a `switch` to lazily `require` and instantiate the matching handler class.
4. **Sanitizes** the tag body, then **splices** the handler's HTML output back into the message with `str_replace($match[0], …)`.

```
switch($match[1]){
  case 'item': case 'itemico': case 'itemdkp':
  case 'ptritem': /* … */
      $object = new bbtips_item($args, $match[1]); break;
  case 'spell': $object = new bbtips_spell($args); break;
  case 'quest': $object = new bbtips_quest($args); break;
  /* …craft, itemset, achievement, npc, wowchar… */
}
$object->ptr = (substr($match[1],0,3) == 'ptr');           // PTR realm?
$message = str_replace($match[0], $object->parse($nameout), $message);
```

The `ptr` prefix is stripped and recorded as a boolean, so a handler transparently targets either the live database or the Public Test Realm domain.

5.3 Class hierarchy

Every tag handler extends the abstract `bbtips` base, which centralizes URL building and the resilient HTTP fetch. The subclass only knows how to *interpret* the response for its object type.

```
abstract bbtips # make_searchurl(), gethtml(), _read_php(), ReplaceWildcards(), NotFound()
├── bbtips_item # item / itemico / itemdkp (XML + JSON name search) ├── bbtips_spell # spell
/ recipe / glyph / guild-perk ├── bbtips_quest ├── bbtips_craft ├── bbtips_itemset ├──
bbtips_achievement ├── bbtips_npc └── wowcharacter # armory profile
```

6 Fetching & caching Wowhead data

6.1 Resolution order (cache-first)

Inside `bbtips_item::parse()` the handler tries the cheapest source first and only falls through to progressively more expensive lookups:

1 · DB
Cache hit?
Fuzzy `LIKE` match on name/search_name for this type+lang+size.
`bbtips_cache::getObject()`

→

2 · XML
Wowhead XML
`/item=NAME&xml`; parsed with SimpleXML, honoring a single-result redirect.
`_getItembyXML()`

→

3 · JSON
Search scrape
Scrapes the Listview JSON off the search page; matches the exact name.
`_getItemByName()`

On a successful remote resolve, the object is written back with `saveObject()` so the next render is a pure cache hit. A total failure yields a `<span class="notfound">` marker rather than a broken tag.

6.2 The resilient fetch

The base class's `_read_php()` is deliberately defensive because it depends on a third-party site and unpredictable PHP host configs. It tries three transports in order, stopping at the first that returns data:

Transport	Used when	Notes
`cURL`	available (preferred)	Sets a browser user-agent, XML accept headers, 30 s timeout, follows redirects; maps cURL error codes to readable messages.
`file_get_contents`	cURL empty & `allow_url_fopen` on	Inspects `$http_response_header` for 200/403/500/503 to classify Wowhead/Armory outages.
`fsockopen`	both above failed	Manual HTTP/1.0 request; strips headers and extracts the XML body between `<?xml` and `</page>`.

> **External-dependency risk.** Resolution reaches out to `wowhead.com` synchronously during page render. The cache is what keeps this from being a per-view cost, and the layered transports + `NotFound()` fallback keep a Wowhead outage from breaking the page — a hard requirement noted repeatedly in the changelog (e.g. 0.3.6 "independent of wowhead status").

6.3 The cache table

`bbtips_cache` is a thin DAL over the resolved-object table. `getObject()` uses phpBB's DBAL portably (`sql_like_expression`, `sql_build_array`) so it runs on MySQL, MSSQL and PostgreSQL. Rows key on *name/search_name + type + lang + size*, storing the numeric `itemid`, quality, rank and icon path — exactly the fields the output templates need.

7 Templates — the HTML "pattern" system

bbTips keeps its output markup out of PHP entirely. Each object type has a tiny HTML file in the active style containing `{wildcards}`. The `wowhead_patterns` class slurps the whole `bbtips/` template folder into a name→markup map at construction:

```
// wowhead_patterns.php — read every *.html into $patterns[name]
while(false !== ($file = readdir($opendir))){
  if(substr($file, strpos($file,'.')+1) == 'html'){
    $this->patterns[$filename] = @file_get_contents($path . $file);
  }
}
```

A handler picks a pattern by name and fills it via `ReplaceWildcards()`, which maps an info array onto tokens like `{link} {name} {qid} {icon} {gems}`:

ITEM.HTML (TEXT LINK)

```
<a class="q{qid}" href="{link}"
   target="_blank">{name}</a>
```

Becomes e.g. `<a class="q4" href="//wowhead.com/item=50468">Sanctified Lightsworn Battleplate</a>`. The `q{qid}` class is what colors the link by item quality.

ICON_MEDIUM.HTML (ICON LINK)

```
<div class="iconmedium"><div
  class="tile"><a href="{link}"
  target="_blank"><img src="{icon}"
  alt="Image" /></a></div></div>
```

Same `{link}` contract — so the same client engine lights it up — but rendered as a WoW icon tile instead of a text link.

> **Why this matters for the pop-up.** Notice both patterns emit an anchor whose `href` is `/item=NNNN`. That URL *is* the entire interface to the client engine. Adding a new visual style for a tooltip means adding an HTML pattern — no PHP and no JS change — as long as the anchor keeps the Wowhead-shaped `href`.

Because patterns are read from `$user->theme['template_path']`, each installed style can ship its own look (prosilver and subsilver2 are both supported), and the base class resolves the icon size (`small`/`medium`/`large`) into the chosen pattern name at runtime.

8 Client side — rendering the pop-up

8.1 Getting the engine onto the page

The installer injects one line into the style's `overall_header.html`: `<!-- INCLUDE bbtips/bbtips_header.html -->`. That partial decides, per the `S_BBTIPS_REMOTE` flag, whether to load Wowhead's hosted engine or the local bundle:

```
<!-- IF S_BBTIPS_REMOTE -->
<script src="http://static.wowhead.com/widgets/power.js"></script>
<!-- ELSE -->
<script>var template_path="{T_SUPER_TEMPLATE_PATH}/bbtips"; var theme_path="{T_THEME_PATH}";</script
<script src="{T_SUPER_TEMPLATE_PATH}/bbtips/power.js"></script>
<!-- ENDIF -->
```

`power.js` depends on `basic.js`, a vendored copy of Wowhead's `$WH` utility library (DOM helpers, cookie/URL/array utilities, and the tooltip scaling math). Together they are the unmodified Wowhead widget, hosted locally so the forum is not hard-tied to Wowhead's CDN.

8.2 One listener, lazy work

On init, the engine attaches a **single** `mouseover` handler to the whole document — it does not pre-scan every link. Work happens only when the user actually hovers something:

```
function attachEvent(){ $WH.aE(document, "mouseover", onMouseOver); }

function onMouseOver(e){
  var t = e._target, i = 0;
  // walk up to 5 ancestors looking for a matching anchor
  while(t != null && i < 5 && scanElement(t, e) == -2323){ t = t.parentNode; ++i; }
}
```

8.3 Recognizing the anchor

`scanElement()` is the mirror image of the server's template pattern: it regex-matches the anchor's `href` against the Wowhead URL shape, extracting the type name and numeric id, then parses any inline modifiers (`gems`, `ench`, `lvl`, `bonus`…):

```
m = t.href.match(
   /wowhead\.com\/\??(item|quest|spell|achievement|npc|itemset|…)=([0-9]+)/);
// m[i1] = "item"  →  numeric type id via $WH.g_getIdFromTypeName()
// m[i2] = "50468" →  the object id
display(type, typeId, locale, params);   // → request() → showTooltip()
```

The type string is converted to Wowhead's numeric type id (item=3, spell=6, quest=5, npc=1, achievement=10, …) and, together with the id, forms the lookup key for the tooltip data.

8.4 Fetch-once, cache, paint

`display()` checks an in-memory table `LOOKUPS[type][id]`. If the tooltip is already `STATUS_OK` it paints immediately; otherwise it shows a "loading" box and fires a one-time `request()`. When data arrives,

`showTooltip()` builds the floating DOM, `$WH.Tooltip.show()` positions it at the cursor, and `onMouseOut()` hides it.

Function	Role in the pop-up
`onMouseOver / scanElement`	Detect a Wowhead-shaped anchor under the cursor.
`display / initElement`	Look up cached tooltip state; decide fetch vs. show.
`request`	Lazily retrieve the tooltip payload exactly once per id.
`showTooltip`	Compose the tooltip HTML (quality color, icon, stats) and hand it to `$WH.Tooltip`.
`onMouseMove / onMouseOut`	Track the cursor to reposition, and tear the pop-up down on exit.

> **Optional link decoration.** If a page defines `wowhead_tooltips` options, the engine can also rewrite link text, prepend a tiny icon (`iconizelinks`), or apply the quality color class (`colorlinks`) — all client-side embellishments on top of the anchor the server produced.

9 Security, limits & configuration

9.1 Guarding the tag body

Because tag contents originate from forum users, the parser sanitizes before rendering. Every tag body is run through `html2txt()`, which strips `<script>`, style blocks, all HTML tags, comments/CDATA, and even the literal `http` token. If sanitizing changed the string, the tag is replaced with an explicit refusal instead of being resolved:

```
if($nameout != $namein){
  $message = str_replace($match[0],
    '<span class="notfound">Illegal HTML/JavaScript found.</span>', $message);
}
```

9.2 Bounding the work

- **Parse cap.** `bbtips_maxparse` limits tags parsed per message, hard-capped at 600 regardless of the admin setting — a guard against a post with hundreds of tags each triggering a remote lookup.
- **Install gate.** The hook returns immediately if `bbdkp_plugin_bbtips_version` is empty, so an uninstalled MOD adds zero render cost.
- **Board gate.** Hooks are only registered when the board is not disabled.

9.3 Admin-facing configuration

Config key	Effect
bbtips_localjs	Load the local `power.js` vs. Wowhead's hosted copy (drives `S_BBTIPS_REMOTE`).
bbtips_lang	Wowhead locale/domain (`en`, `de`, `fr`, …) used for both lookups and tooltips.
bbtips_maxparse	Max tags resolved per message (≤ 600).

Configured through an ACP module (`acp_dkp_bbtooltips`) that installs under the bbDKP "Raids" section when bbDKP is present, or the MODS tab otherwise.

10 Design observations

STRENGTHS

- **Clean seam.** The Wowhead-shaped anchor is the only contract between halves; each side evolves independently.
- **Two-layer caching** keeps both the remote lookups and the tooltip fetches to at most once.
- **Template-driven output** makes new looks a data change, not a code change.
- **Defensive I/O** survives Wowhead outages and varied PHP host configs.

CONSTRAINTS & RISKS

- **Synchronous external calls** at render time on cache misses (mitigated by the DB cache).
- **Scraping coupling** — name search depends on Wowhead's HTML/JSON, which the changelog shows breaking repeatedly.
- **String-concatenated SQL** in the cache layer (names are escaped, but it predates parameter binding).
- **phpBB 3.0 hook API** — the interception mechanism is legacy; a 3.1+/3.3 port would use events/extensions.

One-sentence architecture. bbTips is a render-time PHP text-rewriter that turns game BBCodes into cached, quality-colored Wowhead links, paired with a locally-hosted Wowhead widget that turns those links into hover pop-ups — two decoupled halves joined by a single URL convention.